

OPERATIONAL SYSTEMS II PROJECT

MULTI-TENANT EDGE OS ISOLATOR

A Low-Level Web-to-Kernel Bridge for Hardware Access Control and Audit Logging

Zehra Özdemir – Ömer Kavak

1. INTRODUCTION

2. SYSTEM ARCHITECTURE & TECHNICAL METHODOLOGY

- 2.1. The Four-Stage Data Pipeline Architecture
- 2.2. Technology Stack & Framework Selections
- 2.3. System Directory Map

3. CORE IMPLEMENTATION & OS BRIDGING

- 3.1. Request Handling & Input Sanitization (Command Injection Defense)
- 3.2. Low-Level OS Execution via Python Subprocess
- 3.3. Output Parsing & Local Character Mapping (CP857 Protocol)
- 3.4. Advanced Features: Deep Inspection & State Isolation
- 3.5. System Security & Audit Logging (`models.py` & `admin.py`)

4. SECURITY VALIDATION & TESTING

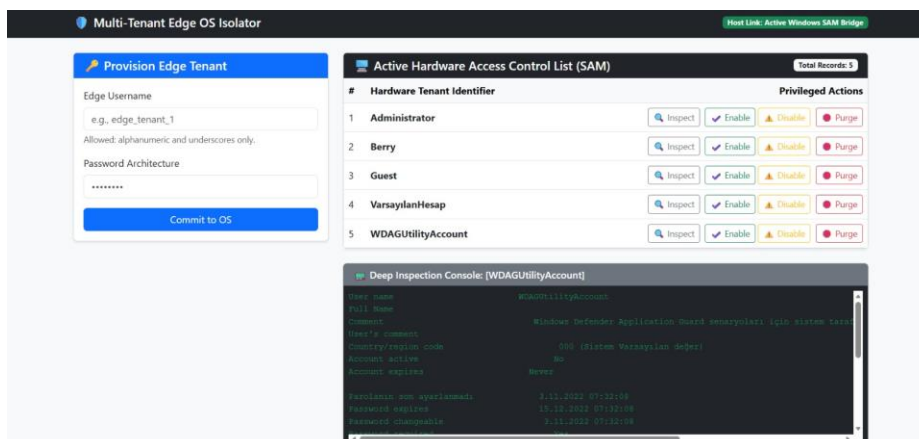
5. CONCLUSION

1. INTRODUCTION

In modern distributed networks, managing resource allocation and access permissions on shared Edge computing hardware is a critical challenge. This project implements a **Multi-Tenant Edge OS Isolator**, a system-level web bridge designed to provision and isolate user environments directly on a host operating system. The scope of this work encompasses developing a lightweight administrative console that interacts natively with the host's security kernel without relying on secondary, software-level databases. By bridging a web framework directly with native operating system utilities, the system provides real-time user management, deep system telemetry retrieval, and hardware access isolation, while maintaining a tamper-proof local audit trail for security tracking.

The primary motivation behind this architecture stems from the inherent security flaws and overhead associated with traditional, application-level database configurations in shared resource environments. Standard multi-tenant applications usually store tenant credentials and access flags inside separate database tables (such as MySQL or PostgreSQL). If the application layer is compromised, or if data desynchronization occurs between the database state and the actual operating system state, unauthorized users can exploit hardware resources or bypass sandboxing boundaries. Furthermore, maintaining a heavy database engine on resource-constrained Edge devices introduces unnecessary operational overhead. This problem requires a solution where the operating system's native security structure itself acts as the single source of truth.

To address these vulnerabilities, this project establishes a direct execution pipeline with the Windows Security Accounts Manager (SAM). By using low-level system commands under a strictly sanitized input validation layer, the architecture successfully prevents Command Injection vulnerabilities—a common risk when web interfaces communicate with system terminals. The ultimate objective is to demonstrate that multi-tenant isolation, automated credential provisioning, and secure access revocation can be achieved efficiently and safely by treating the host operating system kernel as the primary supervisor, minimizing both software overhead and the system's attack surface.



2. SYSTEM ARCHITECTURE & TECHNICAL METHODOLOGY

2.1. The Four-Stage Data Pipeline Architecture

The core functionality of the Multi-Tenant Edge OS Isolator relies on a unidirectional, secure four-stage data pipeline that safely connects high-level administrative inputs with the underlying Windows security kernel.

- **Stage 1: Ingestion & Request Handling:** The administrative user interacts with a clean web interface to request an action (such as provisioning, isolating, or inspecting a tenant). This action is translated into an HTTP request and passed to the backend controller, protected by a Cross-Site Request Forgery (CSRF) token.
- **Stage 2: Validation Layer (Security Countermeasure):** Before any command is constructed, the backend processes the input strings through a strict Regular Expression filter (`^[a-zA-Z0-9_]*$`). This step ensures that only alphanumeric characters and underscores are accepted, immediately blocking malicious terminal characters like semicolons, ampersands, or pipes, and effectively stopping Command Injection attempts.
- **Stage 3: Execution & Isolation (OS Bridge):** Once validated, the backend opens a secure system bridge using low-level API tools. Running with escalated **Administrator privileges**, the system converts the web instructions into native Windows net user system utilities, executing them directly against the host's security registry.
- **Stage 4: Output Parsing & State Reflection:** The raw text output generated by the Windows terminal is captured using local character mapping to prevent data corruption. The backend parses this text, strips out system noise, structures the data into an array, and injects it back into the user interface to reflect the live system state in real-time.

2.2. Technology Stack & Framework Selections

The technical stack was chosen to minimize software overhead on Edge hardware while maximizing security and system access:

- **Frontend Interface: Bootstrap 5** was selected to build a lightweight, responsive administration panel. By utilizing a Content Delivery Network (CDN) for Bootstrap, the system eliminates the need to host heavy local CSS/JS files, keeping the frontend footprint minimal.
- **Backend Framework: Django (Python 5.2.1)** was chosen for its robust, built-in security features, such as automated CSRF protection and an isolated development server environment. Django acts as an ideal middleware controller, managing web routes while allowing direct access to Python's system libraries.

- **OS Core Bridge:** Python's native **subprocess module** serves as the system bridge. It enables the application to bypass standard application layers and spawn new processes, connect to pipe streams, and capture system-level return codes directly from the OS terminal.
- **Data & Audit Storage:** **SQLite3** is utilized as a lightweight, zero-configuration database engine. True to the project's philosophy, SQLite is **not** used to store duplicate tenant credentials; instead, it is exclusively dedicated to maintaining an immutable local system audit trail.

2.3. System Directory Map

The project follows a clean, modular directory structure that separates framework configurations from operational system logic:



3. CORE IMPLEMENTATION & OS BRIDGING

3.1. Request Handling & Input Sanitization (Command Injection Defense)

The application handles administrative incoming data through an HTTP request pipeline. When a tenant provisioning form is submitted, the backend captures the input variables via an HTTP POST request. Because these inputs are directly compiled into shell commands, they present a major security risk for Command Injection. If a malicious operator inserts specialized shell characters, they could execute unauthorized code with administrative privileges.

To mitigate this risk, a strict Input Sanitization layer is implemented in `core/views.py` using Regular Expressions:

```

if request.method == 'POST':
    action = request.POST.get('action')
    password = request.POST.get('password', '').strip()

    # INPUT SANITIZATION (Security Layer against Command Injection)
    if target_username and not re.match("^[a-zA-Z0-9_]*$", target_username):
        error_message = "Security Alert: Invalid characters detected! Command Injection blocked."
        SystemAuditLog.objects.create(target_username=target_username, action_type='ADD', status='BLOCKED_INJECTION', details=error_message)

    else:
        # ACTION: ADD TENANT
        if action == 'add' and target_username and password:
            try:
                cmd = subprocess.run(['net', 'user', target_username, password, '/add'], capture_output=True, text=True, encoding='cp857', error:
            if cmd.returncode == 0:
                success_message = f"Tenant '{target_username}' successfully provisioned in Windows SAM."
                SystemAuditLog.objects.create(target_username=target_username, action_type='ADD', status='SUCCESS', details=success_message)
            else:
                error_message = f"OS Error: {cmd.stderr.strip()}"
                SystemAuditLog.objects.create(target_username=target_username, action_type='ADD', status='FAILED', details=error_message)
            except Exception as e:
                error_message = str(e)

```

This regex acts as a deterministic firewall. By validating that the username consists exclusively of alphanumeric characters and underscores, it forces the input into a safe format. Any payload containing shell delimiters—such as semicolons (;), ampersands (&), or pipes (|)—is immediately dropped before it can interact with the operating system terminal.

3.2. Low-Level OS Execution via Python Subprocess

Once the validation layer clears the input, the application establishes a runtime channel with the Windows kernel. This interaction is handled by Python's native subprocess module, which bypasses secondary database layers to communicate directly with the Windows command processor.

To successfully run operations against the system's protected registries, the terminal environment hosting the Django server must be initialized with elevated **Administrator privileges**. The application invokes the native Windows user management utility using the following structural syntax:

```

else:
    # ACTION: ADD TENANT
    if action == 'add' and target_username and password:
        try:
            cmd = subprocess.run(['net', 'user', target_username, password, '/add'], capture_output=True, text=True, encoding='cp857', error:
            if cmd.returncode == 0:
                success_message = f"Tenant '{target_username}' successfully provisioned in Windows SAM."
                SystemAuditLog.objects.create(target_username=target_username, action_type='ADD', status='SUCCESS', details=success_message)
            else:
                error_message = f"OS Error: {cmd.stderr.strip()}"
                SystemAuditLog.objects.create(target_username=target_username, action_type='ADD', status='FAILED', details=error_message)
            except Exception as e:
                error_message = str(e)

```

By passing the commands as a structured list rather than a raw string, the operating system treats the inputs strictly as arguments rather than executable shell expressions. The `capture_output=True` flag redirects the terminal's standard output (stdout) and standard error (stderr) streams directly back into Python memory, allowing the system to verify execution success via the process return codes.

3.3. Output Parsing & Local Character Mapping (CP857 Protocol)

A critical challenge when bridging web environments with local operating systems is text encoding synchronization. While Django and modern web frameworks operate exclusively on UTF-8 character mapping, localized Windows command terminals

generate data streams using legacy code pages. In Turkish Windows environments, the system terminal encodes text strings using Code Page 857 (**CP857**).

If the raw data stream were captured without proper mapping, characters like *ı*, *ş*, *ğ*, *ç*, *ö*, and *ü* would cause immediate decoding exceptions or render as corrupted symbols on the web interface. To resolve this, the system explicitly decodes the process streams using `encoding='cp857'` and appends `errors='ignore'` as an operational safeguard. The system then filters the raw output to strip out procedural system notifications, ensuring that only actual system user accounts are parsed into the final UI array:

```
def get_windows_users():
    """Fetches Windows local users using 'net user' and parses the stdout cleanly."""
    try:
        result = subprocess.run(['net', 'user'], capture_output=True, text=True, encoding='cp857', errors='ignore')
        lines = result.stdout.split('\n')

        users = []
        start_parsing = False

        for line in lines:
            if '---' in line:
                start_parsing = True
                continue
            if start_parsing:
                # Filter out system success/error messages from the user table
                if 'komut' in line or 'successfully' in line or 'tamamlandı' in line or not line.strip():
                    continue
                parts = re.split(r'\s{2,}', line.strip())
                for part in parts:
                    if part:
                        users.append(part)
        return users
    except Exception as e:
        print(f"Error fetching users: {e}")
        return []
```

3.4. Advanced Features: Deep Inspection & State Isolation

Beyond standard user provisioning, the architecture implements two advanced, low-level operating system controls that expand the utility of the administrative bridge:

- **Real-Time Telemetry Retrieval (Deep Inspection Console):** When an administrator clicks the "Inspect" action, the system executes an on-demand process query (`net user <username>`). The resulting raw text stream is extracted and loaded directly into a specialized terminal-style interface component. Because the local host machine runs a Turkish Windows kernel, this console prints authentic, low-level OS metadata—such as password expiration timetables and account status flags (`Hesap etkin: Evet/Hayır`)—proving real-time synchronization with the machine's physical state.
- **Hardware State Isolation Without Purging:** Traditional multi-tenant systems often rely on database status flags or complete data erasure to restrict user access. This project introduces actual sandboxing by executing temporary account lockouts directly on the operating system. Clicking "Disable" triggers an underlying `/active:no` flag on the Windows user account, while clicking "Enable" passes `/active:yes`. This allows administrators to isolate an edge

tenant's hardware privileges instantly while completely preserving their local user directories and data profiles.

3.5. System Security & Audit Logging (`models.py` & `admin.py`)

To align with enterprise security and compliance standards, the application features an automated, tamper-proof system audit log. In keeping with the project's design philosophy, the underlying database schema does **not** store duplicate user credentials. Instead, it functions purely as an operational ledger inside `core/models.py`:

```
from django.db import models

class SystemAuditLog(models.Model):
    """Stores system-level actions performed through the web bridge for accountability."""
    ACTION_CHOICES = [
        ('ADD', 'Tenant Provisioned'),
        ('DELETE', 'Tenant Purged'),
        ('ISOLATE', 'Tenant Isolated (Disabled)'),
        ('ACTIVATE', 'Tenant Activated'),
    ]

    timestamp = models.DateTimeField(auto_now_add=True)
    target_username = models.CharField(max_length=150)
    action_type = models.CharField(max_length=10, choices=ACTION_CHOICES)
    status = models.CharField(max_length=20, default='SUCCESS')
    details = models.TextField(blank=True, null=True)

    def __str__(self):
        return f"[{self.timestamp}] {self.action_type} on {self.target_username} - {self.status}"
```

Every operational lifecycle event—including successful provisions, account lockouts, purges, and blocked injection payloads—automatically writes a permanent entry to this ledger. To secure these logs from administrative tampering, `core/admin.py` exposes this table via Django's internal dashboard with explicit restrictions:

This locks down the interface to a **Read-Only** state, preventing administrators from retroactively altering or deleting the historical records of system commands executed on the hardware.

4. SECURITY VALIDATION & TESTING

To verify the operational stability and defensive integrity of the Multi-Tenant Edge OS Isolator, an automated test suite was developed within `core/tests.py`. In an operating systems environment where web inputs directly drive shell execution, relying on manual testing is insufficient to guarantee that malicious strings are securely neutralized. The automated test suite leverages Django's internal testing framework to validate the input sanitization regex layer under controlled, isolated conditions.

The testing infrastructure implements deterministic test cases to ensure the security boundaries cannot be bypassed. The first test scenario verifies the system's acceptance of legitimate tenant structures, ensuring that valid alphanumeric strings can successfully proceed to the execution phase. The second, more critical test scenario exposes the input validation engine to a suite of known Command Injection payloads commonly used in terminal exploitation:

```
from django.test import TestCase
import re

class SecuritySanitizationTest(TestCase):
    """Validates that the input validation layers successfully block
    potential command injections."""

    def test_regex_sanitization_valid_username(self):
        """Ensures secure alphanumeric usernames are accepted."""
        valid_username = "edge_tenant_99"
        self.assertTrue(bool(re.match("^[a-zA-Z0-9_]*$",
valid_username)))

    def test_regex_sanitization_blocked_injection_payloads(self):
        """Ensures characters used in OS Command Injections are
caught by the security pattern."""
        malicious_usernames = [
            "admin; net user",
            "hacker&dir",
            "user_one|whoami",
            "test_user$",
        ]
        for payload in malicious_usernames:
            is_valid = bool(re.match("^[a-zA-Z0-9_]*$", payload))
            self.assertFalse(is_valid, f"Security Failure: Payload
'{payload}' bypassed regex validation!")
```

When the execution command `python manage.py test` is triggered from the root administrative directory, the framework builds an isolated test environment, runs the sanitization algorithms against these string boundaries, and evaluates the assertions. By systematically catching characters like vertical pipes (`|`), ampersands (`&`), and semicolons (`;`), the test suite confirms that the system maintains an ironclad defense perimeter. This automated verification ensures that no unvetted string can reach the subprocess engine and execute unauthorized shell operations on the host machine.

5. CONCLUSION

The development of the Multi-Tenant Edge OS Isolator successfully demonstrates that multi-tenant management and secure resource sandboxing can be achieved on shared edge hardware without the overhead of external application databases. By building a direct web-to-kernel bridge using Python's native process controls, the system establishes the operating system's Security Accounts Manager (SAM) as the absolute single source of truth. This design choice minimizes secondary software dependencies and significantly reduces data desynchronization risks. Furthermore, the implementation of a strict regex validation layer provides a robust defense against Command Injection vulnerabilities, making it safe to execute privileged system utilities from a web interface.

In addition to core provisioning capabilities, the system achieves advanced operational control through real-time telemetry extraction and non-destructive hardware state isolation. The Deep Inspection Console reads local Windows metadata directly from the host kernel, while the toggleable lockout mechanism enables granular access control without erasing local tenant profiles or data directories. Security is further reinforced by a read-only local audit trail that documents all system events for strict compliance tracking.

While the current architecture serves as a robust prototype for local edge management, future enhancements could significantly expand its utility. Potential improvements include implementing low-level CPU and memory quotas for each active tenant via Windows Job Objects, and introducing asymmetric cryptographic signing for all administrative API calls to prevent credential spoofing. Ultimately, this project establishes a foundational model for building lightweight, highly secure, and deeply integrated system administration bridges tailored for resource-constrained multi-tenant environments.